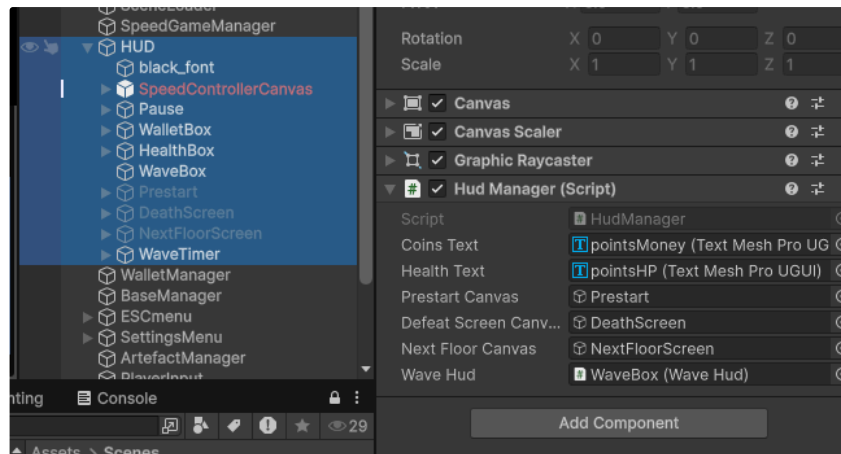


- Чтобы некоторые элементы интерфейса чувствовали себя более комфортно и работать с ними было тоже куда более проще - они были собраны в один единый HUD и вынесены в Persistent сцену. HUD просто включается и выключается когда нужно (его вызывает GameFlowManager).



Верхняя панель интерфейса.



Что входит в HUD. Hud Manager.

- Я также сделал дочерними PrestartScreen, DeathScreen, NextFloorScreen. Потому что это позволит мне более деликатно ими управлять и более того, они появляются только при включенном HUD. Тот же SettingsController не является дочерним, потому что может быть вызван как в игре, когда HUD включен, так и из Main\_Menu, когда HUD выключен.

В Awake() я получаю все нужные скрипты с дочерних компонентов:

```
1 using TMPro;
2 using Unity.IO.LowLevel.Unsafe;
3 using UnityEngine;
4 using UnityEngine.InputSystem;
5
6 public class HudManager : Manager<HudManager>
7 {
8     [SerializeField] private TMP_Text coinsText;
9     [SerializeField] private TMP_Text healthText;
10
11     [SerializeField] private GameObject prestartCanvas;
12     [SerializeField] private GameObject defeatScreenCanvas;
13     [SerializeField] private GameObject nextFloorCanvas;
14
15     [SerializeField] private GameObject timerWaveCanvas;
16     [SerializeField] private TextMeshProUGUI timerTXT;
17 }
```

```

18     [SerializeField] private WaveHud waveHud;
19
20     private PrestartUI prestartUI;
21     private DefeatScreen defeatScreen;
22     private NextFloorScreen nextFloorScreen;
23
24     public bool IsHudOn { get; private set; } = false;
25
26     protected override void Awake()
27     {
28         base.Awake();
29
30         defeatScreen = defeatScreenCanvas.GetComponent<DefeatScreen>();
31         prestartUI = prestartCanvas.GetComponent<PrestartUI>();
32         nextFloorScreen = nextFloorCanvas.GetComponent<NextFloorScreen>();
33         if (waveHud == null) { waveHud = GetComponent<WaveHud>(); }
34
35         HideDefeatScreen();
36     }

```

После чего в Start() я вызываю Reset(), делаю подписки и скрываю сам менеджер. Правильнее было бы держать скрипт родительским объектом, а главный канвас поместить как дочерний и выключать его, чтобы менеджер всегда жил. Моё текущее решение может вызвать в редких случаях некоторые проблемы. Поэтому возможно я в дальнейшем изменю его на более правильный вариант.

Методы обновления UI принимают данные и отображают их на экране.

Show() - включает объект и обновляет поля (на случай, если данные обновятся).

Hide() - скрывает объект.

```

1     private void Start()
2     {
3         Reset();
4
5         BaseManager.Instance.OnHealthChanged += UpdateHealthUI;
6         WalletManager.Instance.OnCoinsChanged += UpdateCoinsUI;
7
8         prestartUI.OnPressed += CallStartPlaying;
9
10        Hide();
11    }
12
13    private void UpdateHealthUI(float x, float health)
14    {
15        healthText.text = health.ToString();
16    }
17
18    private void UpdateCoinsUI(int coins)
19    {
20        coinsText.text = coins.ToString();
21    }
22
23    public void Show()
24    {
25        IsHudOn = true;
26        this.gameObject.SetActive(true);
27        UpdateInfoFields();
28    }
29
30    public void Hide()
31    {
32        IsHudOn = false;
33        this.gameObject.SetActive(false);
34    }

```

Можно показать или скрыть окно пре-старта, окно экрана смерти, окно завершения этажа.

На случай, если объект всё же будет уничтожен - OnDestroy (хотя объект является синглтоном и не должен уничтожаться в течении всего цикла игры).

UpdateFields - подтягивает информацию из менеджеров.

Для работы с отображением волн ещё два метода - установить максимум и установить текущую волну.

```
1 public void HidePrestart()
2 {
3     prestartUI.Hide();
4     prestartCanvas.SetActive(false);
5 }
6
7 public void ShowPrestart()
8 {
9     prestartCanvas.SetActive(true);
10 }
11
12 public void ShowPrestartButton()
13 {
14     prestartUI.Show();
15 }
16
17 public void ShowDefeatScreen()
18 {
19     defeatScreen.ShowStatsRun();
20     defeatScreenCanvas.SetActive(true);
21 }
22
23 public void HideDefeatScreen()
24 {
25     defeatScreenCanvas.SetActive(false);
26 }
27
28 public void ShowStageCompletedCanvas()
29 {
30     nextFloorScreen.Show();
31 }
32
33 private void OnDestroy()
34 {
35     if (BaseManager.Instance != null)
36         BaseManager.Instance.OnHealthChanged -= UpdateHealthUI;
37
38     if (WalletManager.Instance != null)
39         WalletManager.Instance.OnCoinsChanged -= UpdateCoinsUI;
40
41     //prestartUI.OnPressed -= CallStartPlaying;
42 }
43
44 private void CallStartPlaying()
45 {
46     GameFlowManager.Instance.StartPlaying();
47 }
48
49 private void UpdateInfoFields()
50 {
51     coinsText.text = WalletManager.Instance.Coins.ToString();
52     healthText.text = BaseManager.Instance.MaxHp.ToString();
53 }
54
55 public void Reset()
56 {
57     HideDefeatScreen();
58     nextFloorScreen.Hide();
59     ShowPrestart();
60     UpdateInfoFields();
61 }
62
63 public void SetTotalWaveIcons(int totalwave)
64 {
65     waveHud.Init(totalwave);
66 }
67
68 public void SetCurrentWaveIcon(int wave)
69 {
70     waveHud.SetCurrentWave(wave);
71 }
72
73 public void TimerWaveHide()
74 {
75     if (timerWaveCanvas != null)
76         timerWaveCanvas.SetActive(false);
77 }
78
79 public void TimerWaveShow()
```

```

80     {
81         if (timerWaveCanvas != null)
82             timerWaveCanvas.SetActive(true);
83     }
84     public void TimerWaveSet(float time)
85     {
86         if (timerTXT == null) return;
87
88         timerTXT.text =
89             System.TimeSpan.FromSeconds(time)
90                 .ToString(@"mm\:ss");
91     }
92 }

```

## SpeedGameController.

Контролер скорости игры. Более подробно о нём расписано на странице SpeedGameManager. Дублировать сюда всю логику мне не хочется. Вкратце: есть менеджер скорости, который хранит значение множителя. А есть кнопки на интерфейсе - которые меняют это значение. Условный враг при вычитывании скорости движения умножает свою скорость на множитель скорости игры. Никаких Time.timeScale.

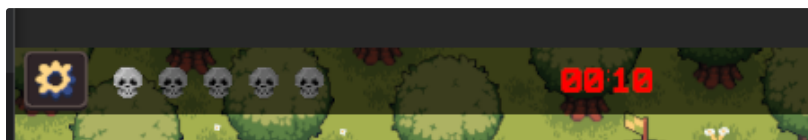
Управление доступно нажатием кнопок и клавишами: 1, 2, 3, 4.



SpeedGameController.

## WaveBox.

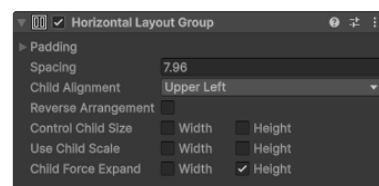
Для отображения всех волн и текущей волны, я сделал область с черепками, где подсвеченные - указывают прошедшее количество, а затемненные - сколько ещё осталось.



WaveBox.

На канвас WaveBox я повесил компонент для группировки элементов. Установил группировку по высоте и сделал 7.96 дистанцию. Теперь под этот канвас можно добавлять любые элементы и они все будут выравниваться в линию.

Я просто буду добавлять sprite черепков, а после пересобирать их. Если волн оказывается 3, то первые 3 будут естественными



Horizontal Layout Group.

спрайтами, а оставшиеся - чуть прозрачнее.

Init() - Инициализируем количество волн и заполняем неактивными спрайтами всю панель. После в SetCurrentWave(int) получаем индекс текущей волны и пересобираем линию спрайтов.

```
1 using System.Collections.Generic;
2 using UnityEngine;
3 using UnityEngine.UI;
4
5 public class WaveHud : MonoBehaviour
6 {
7     [SerializeField] private Transform container;
8     [SerializeField] private Image prefabIcon;
9
10    [SerializeField] private Color activeColor = Color.white;
11    [SerializeField] private Color inactiveColor = new Color(1f, 1f, 1f, 0.5f);
12
13
14    private List<Image> icons = new();
15
16
17    public void Init(int totalWaves)
18    {
19        Clear();
20
21        for(int i = 0; i < totalWaves; i++)
22        {
23            var icon = Instantiate(prefabIcon, container);
24            icon.color = inactiveColor;
25            icons.Add(icon);
26        }
27    }
28
29    public void SetCurrentWave(int waveIndex)
30    {
31        for (int i = 0; i < icons.Count; i++)
32        {
33            icons[i].color = i < waveIndex
34                ? activeColor
35                : inactiveColor;
36        }
37    }
38
39    private void Clear()
40    {
41        foreach (var icon in icons)
42        {
43            Destroy(icon.gameObject);
44        }
45        icons.Clear();
46    }
47 }
```

Рядом на скрине можно заметить красный таймер. Учитывая, что в моей демо не планируется более 5-ти волн, то количество черепков никогда не достигнет его. Однако возможно его стоит перенести в другое место.

#### PrestartScreen.

Не самая обязательная штука, но было решено всё же её добавить. Как только затемнение экрана (переход FadeManager) спадает, то появляется текст: PRESS TO START. Это прозрачная кнопка, растянутая на весь экран, с лёгким затемнением.



Press to Start Button - PrestartScreen.

Нажать на кнопку можно мышкой, а можно - нажатием почти любой клавиши. Я сделал отдельную карту событий на отслеживание доступных клавиш, и добавил обработку такого события в этой класс: `public void OnAnyKey(InputAction.CallbackContext ctx)` - мы получаем калбек при нажатии. Если нажата любая клавиша → вызывается `HandlePress`. Он же вызывается при клике мышкой.

`HandlePress()` - делает кнопку не кликабельной, потому что ранее это вызывало баг - иногда получалось закликать кнопку, чтобы вызвать дублирование логики. Можно было не использовать событие, а просто вызывать `GameFlowManager`, но я решил передать это действие в `HudManager`, который подписан на событие и сам вызывает `GameFlowManager.Instance.StartPlaying();`

```
1 using System;
2 using TMPro;
3 using UnityEngine;
4 using UnityEngine.InputSystem;
5 using UnityEngine.UI;
6
7 public class PrestartUI : MonoBehaviour
8 {
9     [Header("UI Ref")]
10    [SerializeField] private Button prestartButton;
11    [SerializeField] private GameObject prestartText;
12
13    public event Action OnPressed;
14    private bool inputEnabled = false;
15
16    private void Awake()
17    {
18        prestartButton.onClick.AddListener(HandlePress);
19        Hide();
20    }
21
22    public void Show()
23    {
24        prestartText.SetActive(true);
25        prestartButton.interactable = true;
26        inputEnabled = true;
27    }
28
29    public void Hide()
30    {
31        prestartText.SetActive(false);
32        prestartButton.interactable = false;
33        inputEnabled = false;
34    }
35
36    public void HandlePress()
37    {
38        if (!inputEnabled) return;
39        inputEnabled = false;
40        prestartButton.interactable = false;
41
42        OnPressed?.Invoke();
43    }
```

```

44
45     public void OnAnyKey(InputAction.CallbackContext ctx)
46     {
47         if (!ctx.performed) return;
48         HandlePress();
49     }
50 }

```

## DeathScreen.

❗ Экран гибели называть таковым странно, как и называть его экраном поражения, ведь в бесконечной игре просто нельзя победить. Поэтому нас встречает вывеска THE END. А также следующая статистика: очки, убийства, волны, этажи, общее время забега (не ускоренное менеджером скорости).

У игрока есть кнопка Restart, чтобы начать сначала или кнопка выхода.



The End Screen.

Мы просто подгружаем на экран статистику из GameManager'a. А также предоставляем два метода под кнопки. Ультра-простенько...

К слову... Кнопки анимированны примерно также, как и в главном меню. Подробнее на странице Main Menu.

```

1  using TMPPro;
2  using UnityEngine;
3
4  public class DefeatScreen : MonoBehaviour
5  {
6      [SerializeField] private TMP_Text scoreTxt;
7      [SerializeField] private TMP_Text killsTxt;
8      [SerializeField] private TMP_Text wavesTxt;
9      [SerializeField] private TMP_Text floorTxt;
10     [SerializeField] private TMP_Text timeTxt;
11
12     private void Awake()
13     {
14         gameObject.SetActive(false);
15     }
16
17     public void ShowStatsRun() => ShowStats(GameManager.Instance.GetRun
18
19     private void ShowStats(RunStats stats)
20     {
21         scoreTxt.text = stats.score.ToString();
22         killsTxt.text = stats.kills.ToString();
23         wavesTxt.text = stats.waves.ToString();
24         floorTxt.text = stats.floor.ToString();
25         timeTxt.text = TimeFormatter.ToTime(stats.runTime);
26     }
27
28     public void OnRestart() => GameFlowManager.Instance.RestartGame();
29     public void OnLeave() => GameFlowManager.Instance.LeaveGame();

```

## NextFloorScreen (ArtefactsScreen).

Если же игрок пережил все волны, то, в духе рогаликов, нам открывается меню с выбором одного из нескольких предметов. Подробнее о системе предметов на странице Artefacts Systems.

Здесь можно покинуть игру или продолжить. Однако не выбрав предмета - продолжить не получится.

Три бокса с описанием, иконкой и названием предмета - классика. При выборе предмета кнопка SELECT меняется на SELECTED и перекрашивается в зелёный. При выборе другого → прошлая возвращается в исходное состояние, а новая приобретает эффект выбора.



Интерфейс выбора предметов.



Выбранный предмет меняет надпись и подсвечиваемость.

Создаём класс под панели. Храним ссылки на все элементы панели, а также цвета текста. Создаем поле под ArtefactSO - ScriptableObject.

Initialize(ArtefactSO) - инициализируем наш интерфейс данными из пришедшего пакета данных.

Click() - вызывает событие клика. На родительском объекте панели висит IeventTrigger компонент, который обрабатывает клик и вызывает метод Click(). К слову, событие будет обрабатывать отдельный класс, он будет и вызывать SetSelected();

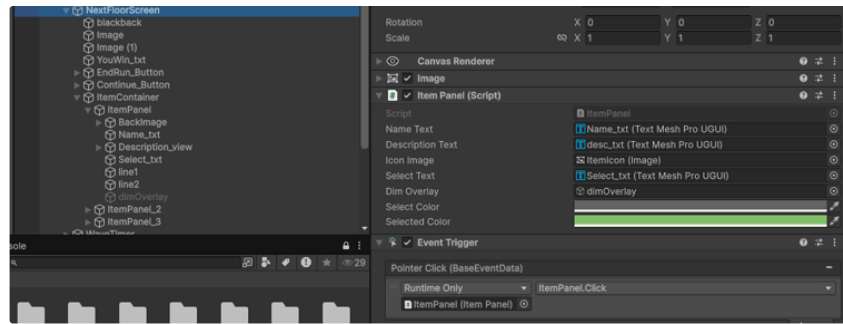


SetSelected() - эффект выбора панели. Меняет цвет и надпись, а также включает затемнение.

GetArtefact() - возвращает сам SO артефакт. Понадобится, когда игрок будет выбирать панель и нажимать продолжить, чтобы применить артефакт.

Clear() - сбрасывает интерфейс и поле артефакта, чтобы повесить следующие.

```
1 using System;
2 using TMPro;
3 using UnityEngine;
4 using UnityEngine.UI;
5
6 public class ItemPanel : MonoBehaviour
7 {
8     public Action<ItemPanel> OnClicked;
9
10    [SerializeField] private TMP_Text nameText;
11    [SerializeField] private TMP_Text descriptionText;
12    [SerializeField] private Image iconImage;
13    [SerializeField] private TMP_Text selectText;
14    [SerializeField] private GameObject dimOverlay;
15
16    [SerializeField] private Color selectColor;
17    [SerializeField] private Color selectedColor;
18
19    private ArtefactSO artefact;
20
21    public void Initialize(ArtefactSO data)
22    {
23        gameObject.SetActive(true);
24        artefact = data;
25        nameText.text = data.name;
26        descriptionText.text = data.description;
27        iconImage.sprite = data.icon;
28
29        SetSelected(false);
30        dimOverlay.SetActive(false);
31    }
32
33    public void Click()
34    {
35        OnClicked?.Invoke(this);
36    }
37
38    public void SetSelected(bool selected)
39    {
40        selectText.text = selected ? "SELECTED" : "SELECT";
41        selectText.color = selected ? selectedColor : selectColor;
42        dimOverlay.SetActive(!selected);
43    }
44
45    public ArtefactSO GetArtefact() => artefact;
46
47    public void Clear()
48    {
49        nameText.text = "";
50        descriptionText.text = "";
51        iconImage.sprite = null;
52        SetSelected(false);
53        selectText.text = "SELECT";
54        selectText.color = selectColor;
55        dimOverlay.SetActive(false);
56        gameObject.SetActive(false);
57    }
58
59 }
```



Иерархия и инспектор.

Создаём ScriptableObject класс, который будет конфигом для артефакта. А также сам эффект тоже делаем SO. Это позволит в артефакт пробрасывать SO настраиваемый, самостоятельный вариант эффекта.

```

1 using UnityEngine;
2
3 [CreateAssetMenu(menuName = "Artefacts/Artefact")]
4 public class ArtefactSO : ScriptableObject
5 {
6     public int IDA;
7     public string artefactName;
8     [TextArea] public string description;
9     public Sprite icon;
10
11     public ArtefactEffectSO effectArtefact;
12 }
13
14 public abstract class ArtefactEffectSO : ScriptableObject
15 {
16     public abstract void Equip();
17     public abstract void Dequip();
18 }

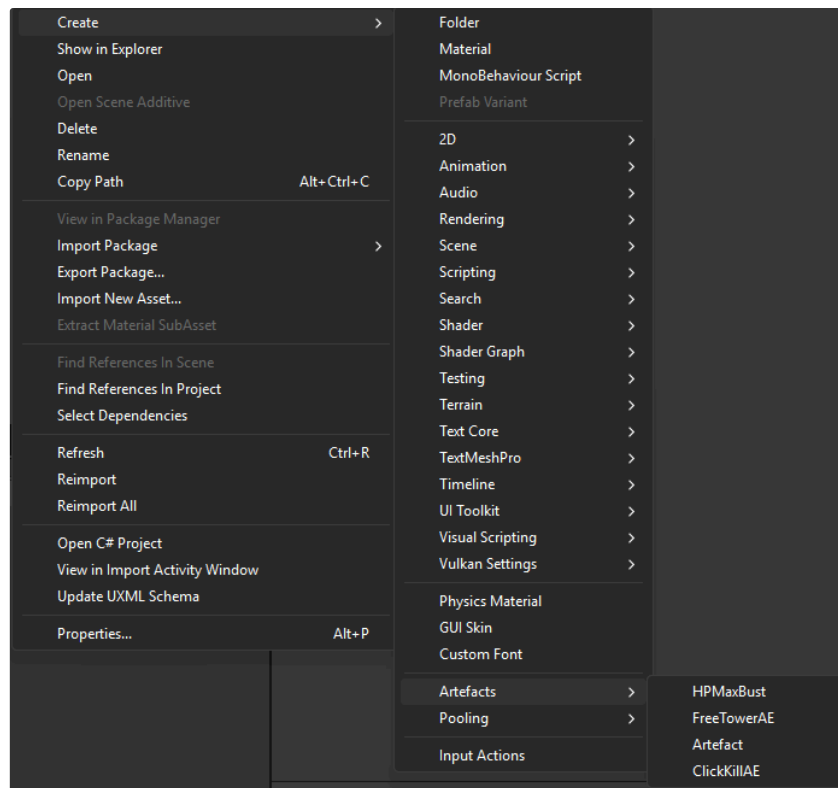
```

Пример предмета:

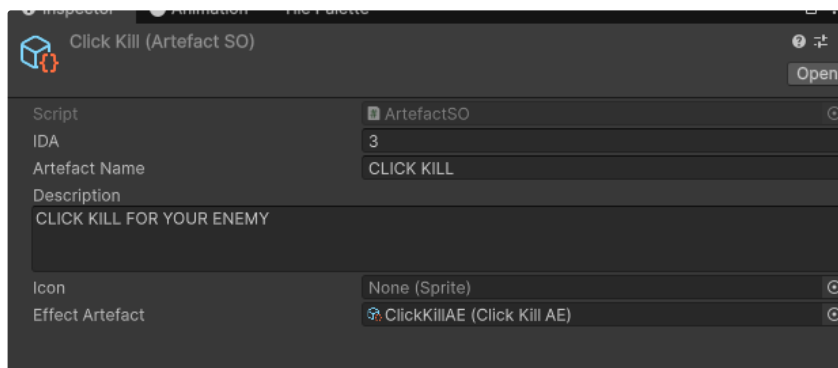
```

1 using UnityEngine;
2
3 [CreateAssetMenu(menuName = "Artefacts/ClickKillAE")]
4 public class ClickKillAE : ArtefactEffectSO
5 {
6     public override void Dequip()
7     {
8         Debug.LogError("Artefact 3 Dequip");
9     }
10
11     public override void Equip()
12     {
13         Debug.LogError("Artefact 3 Equip");
14     }
15 }

```



Создание SO эффекта.



Настройка конфига артефакта.

Теперь к самому скрипту экрана выбора предмета.

Храним лист панелей и выбранный артефакт. Скрываем сами себя и подписываем под событие каждой панели метод ClickedPanel();

Show() - предполагает включение и загрузку случайных предметов в панели. Мы просто будем пробрасывать полученные ArtefactSO внутрь ItemPanel.Initialize(ArtefactSO) и выводить на экран.

```

1 using System;
2 using System.Collections.Generic;
3 using UnityEngine;
4 using UnityEngine.UI;
5
6 public class NextFloorScreen : MonoBehaviour
7 {
8     public Action OnEndRunWinMenu;
9     public Button continueButton;
10
11     [SerializeField] private List<ItemPanel> itemPanels;
12     [SerializeField] private int itemAmount = 3;
13     public int GetItemAmount() => itemAmount;
14     public void SetItemAmount(int amount) => itemAmount = amount;
15 }

```

```

16     private ItemPanel selectedPanel;
17     private ArtefactSO selectedArtefactSO;
18
19
20     private void Awake()
21     {
22         Hide();
23         foreach (var itemPanel in itemPanels)
24         {
25             itemPanel.OnClicked += ClickedPanel;
26         }
27     }
28
29     public void Show()
30     {
31         gameObject.SetActive(true);
32         LoadNewItems();
33     }
34
35     public void Hide() =>
36         gameObject.SetActive(false);

```

ContinueButton() - вызывается нажатием кнопки на интерфейсе. Передаёт в ArtefactManager выбранный предмет и вызывает продолжение игры у GameFlowManager.

Интересный момент - проверки на то, что предмет выбран - нет. Хотя и можно было обойтись флагом, решением стало просто сделать кнопку некликабельной. А как только выбирается хотя бы один предмет (клик по панели вызывает событие и событие вызывает ClickedPanel(передавая себя же)) - то кнопка становится активной. А вернуться к состоянию отсутствия выбора нельзя.

ClearPanels() - вызываем у каждой панели очистку интерфейса, устанавливаем все наши поля на null/false.

LoadNewItems() - очищаем панели, получаем список случайных предметов из менеджера, проверяем на меньшее значение. И инициализируем присутствующие панели. В случае если предметов будет меньше панелей, то панели не инициализируются и не включаются.

ClickedPanel(ItemPanel panel) - реагирует на событие и получает панель, по которой кликнули. Если панель и так выбрана - игнорируем. Если нет - выбираем панель, ищем среди листа панелей выбранную - Активируем и получаем её предмет.

```

1     public void ContinueButton()
2     {
3         if (continueButton == null) return;
4         ArtefactManager.Instance.EquipArtefact(selectedArtefactSO);
5         GameFlowManager.Instance.ContinueAfterArtifact();
6     }
7     public void EndRunButton() =>
8         GameFlowManager.Instance.Defeat();
9
10    public void LoadNewItems()
11    {
12        ClearPanels();
13        var items = ArtefactManager.Instance.GetRandomArtefacts(itemAmount);
14
15        int count = Mathf.Min(items.Count, itemPanels.Count);
16        if (count < 1) {
17            Debug.LogError("Not a single item was received from the ArtefactManager");
18        }
19        for (int i = 0; i < count; i++)
20        {
21            itemPanels[i].Initialize(items[i]);
22        }
23    }
24    private void ClearPanels()
25    {
26        foreach (var item in itemPanels)
27        {
28            item.Clear();
29        }
30        selectedPanel = null;
31        selectedArtefactSO = null;
32        continueButton.interactable = false;

```

```

33
34
35 // Метод не очищает панели как объекты, поэтому отпуски пока не тре
36 // Панели по стандарту будет три. Для игры этого может быть достато
37 // Однако, логику можно будет слегка изменить, чтобы допустить возм
38 // динамически, по ходу забега. В таком случае отпуски следует
39
40
41 private void ClickedPanel(ItemPanel panel)
42 {
43     if (panel == selectedPanel) { return; }
44     selectedPanel = panel;
45     continueButton.interactable = true;
46
47     foreach (var item in itemPanels)
48     {
49         if (selectedPanel == item)
50         {
51             selectedPanel.SetSelected(true);
52             selectedArtefactSO = selectedPanel.GetArtefact();
53         }
54         else item.SetSelected(false);
55     }
56 }
57
58 }

```

⚠ Сейчас NextFloorScreen работает с панелями по ссылкам из инспектора. А количество предметов жестко выставлено в три. Для данного проекта этого достаточно и он не предполагает наличие большего количества. Однако, расширить систему можно очень легко, немного изменив NextFloorScreen, создавая нужное количество панелей или просто переиспользуя большее количество, чем 3. (Даже сейчас можно указать в лист больше панелей, создав их на сцене, просто использоваться будет только 3. Можно будет в таком случае указать случайный дроп 2-5 предметов на выбор, но для меня это не нужно).

Информация про ArtefactManager будет находиться на странице Artefacts System.

## ESC Screen.

📘 В подавляющем большинстве игр есть меню-паузы. Его роль остановить игру, а также предоставить некий функционал: настройки, информацию о частях игры, выход и прочее. При нажатии клавиши ESC (или кнопки на интерфейсе) - у меня в игре вызывается ESC menu.



Esc-Menu.



Esc-Menu: Settings Button.



Повторная проверка кнопки выхода.

Return - возвращает нас в игру, и включает прежнюю скорость. Settings - вызывает меню настроек. End - сначала загорается красным, и лишь при повторном нажатии вызывает экран завершения игры, со всеми статистическими данными.

```

1  using TMPro;
2  using UnityEngine;
3  using UnityEngine.EventSystems;
4  using UnityEngine.InputSystem;
5  using UnityEngine.UI;
6
7  public class ESCScreen : MonoBehaviour
8  {
9      [SerializeField] private GameObject root;
10     [SerializeField] private GameObject endrunButtonObj;
11
12     [SerializeField] private Sprite red_endrunButtonSprite;
13     [SerializeField] private Sprite red_endrunButtonHighlighterSprite;

```

```

14 [SerializeField] private Sprite red_endrunButtonPressedSprite;
15
16 private bool settingsOn = false;
17
18 private Button endrunButton;
19 private TMP_Text endrunText;
20 private Image endrunImage;
21 private Sprite defaultSprite;
22 private SpriteState defaultState;
23 private SpriteState redState;
24
25
26 private bool endrunPress = false;
27
28 private void Start()
29 {
30     endrunText = endrunButtonObj.GetComponentInChildren<TMP_Text>()
31     endrunImage = endrunButtonObj.GetComponent<Image>();
32     defaultState = (endrunButtonObj.GetComponent<Button>()).sprite
33     defaultSprite = endrunImage.sprite;
34     endrunButton = endrunButtonObj.GetComponent<Button>();
35
36     redState = defaultState;
37     redState.highlightedSprite = red_endrunButtonHighlighterSprite;
38     redState.pressedSprite = red_endrunButtonPressedSprite;
39
40     root.SetActive(false);
41 }

```

Hide() - скрывает меню и вызывает ReturnPlaying(), который вызовет восстановление скорости игры.

Show() - показывает меню, вызывая состояние EscMenu, что ставит игру на логическую паузу. Чтобы меню нельзя было вызвать повторно или где-нибудь вне игры, проверяем состояние на Playing.

EscKeyPress - через карту событий обрабатываем нажатие ESC клавиши. Если меню закрыто - открываем, если открыто - закрываем. Помимо этого вызов Show() также находится на кнопке в верхней панели слева.

```

1 public void Hide()
2 {
3     root.SetActive(false);
4     if (endrunPress)
5     {
6         ResetEndButton();
7     }
8     if (settingsOn)
9     {
10         SettingsMenuController.Instance.Hide();
11         settingsOn = false;
12     }
13     GameFlowManager.Instance.ReturnPlaying();
14 }
15
16 public void Show()
17 {
18     if (GameFlowManager.Instance.State != GameState.Playing) return
19     root.SetActive(true);
20     GameFlowManager.Instance.EscMenu();
21 }
22
23 public void EscKeyPress(InputAction.CallbackContext ctx)
24 {
25     if (!ctx.performed) return;
26     if (root.activeSelf)
27     {
28         Hide();
29         if (settingsOn)
30             SettingsMenuController.Instance.Hide();
31     }
32     else
33     {
34         Show();
35     }
36 }
37
38 }

```

ReturnButton - скрываем панель. SettingsButton - вызываем меню настроек. EndRunButton - завершаем забег.

Интересный момент, при закрытии панели важно также закрыть меню настроек, и вернуть кнопке выхода прежнее состояние, если вдруг игрок передумал выходить (подтверждать выход).

```
1 public void ReturnButton() => Hide();
2
3 public void SettingsButton()
4 {
5     if (!SettingsMenuController.Instance.isActiveAndEnabled)
6     {
7         SettingsMenuController.Instance.Show();
8         settingsOn = true;
9     }
10    else if (SettingsMenuController.Instance.isActiveAndEnabled)
11    {
12        SettingsMenuController.Instance.Hide();
13        settingsOn = false;
14    }
15 }
16
17 public void EndRunButton()
18 {
19     if (endrunPress) { EndButton(); return; }
20     endrunPress = true;
21     SetRedButton();
22     endrunText.text = "END";
23 }
24
25 public void EndButton()
26 {
27     Hide();
28     GameFlowManager.Instance.Defeat();
29 }
30
31 public void ResetEndButton()
32 {
33
34     endrunText.text = "ENDRUN";
35     endrunPress = false;
36     SetDefaultButton();
37 }
38
39 private void SetRedButton()
40 {
41     endrunImage.sprite = red_endrunButtonSprite;
42     endrunButton.spriteState = redState;
43 }
44
45 private void SetDefaultButton()
46 {
47     endrunImage.sprite = defaultSprite;
48     endrunButton.spriteState = defaultState;
49 }
50
51 }
```